

Qt Knight 项目总体架构

阅读对象： 所有参与开发、联调和验收的成员
技术基线： C++17、Qt 6.12、MSVC2022、64 位、CMake
文档定位： 项目架构入口。具体结构体和函数见[模块接口协作约定](#)，逐项操作的调用路径见[业务逻辑与数据流](#)。

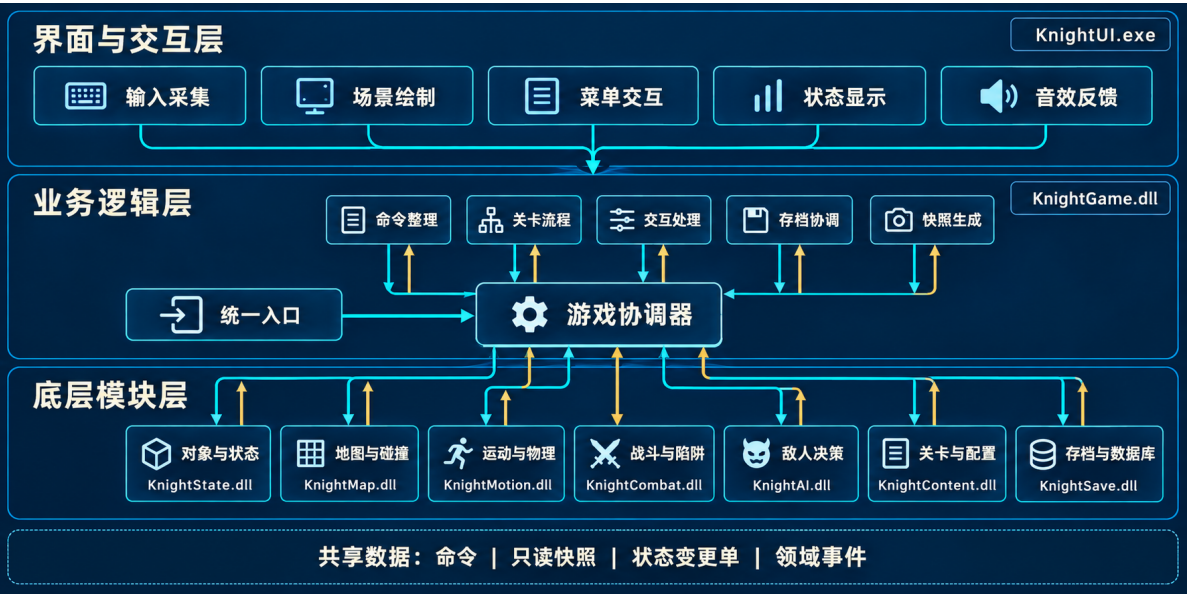
1 项目要做什么

Qt Knight 是一款参考《Hollow Knight》横版探索与战斗体验、使用独立关卡与资源制作的桌面游戏。玩家从菜单进入关卡，在图块地图中移动、跳跃、攻击、躲避陷阱、击败敌人，经过检查点、拾取关键物品和出口进入下一关；死亡后可以复活，进度能够保存并在下次启动时恢复。

最小可玩闭环是： 启动游戏 → 开始或读档 → 进入一关 → 移动、跳跃与攻击 → 受到伤害或触发陷阱 → 激活检查点 → 到达出口 → 保存进度 → 重新打开游戏并继续。联调时先贯通这一关，再扩展到三关、两类陷阱、两类敌人行为和关键物品门。视觉风格与界面排版可以后续细化，但这些操作与反馈必须在架构中有明确归属。

本项目的关键工程目标有三个：**玩法数据只有一个权威来源**；**七个底层模块可以独立实现和构建**；**一次用户操作能够追踪到明确的模块调用、状态提交和界面反馈**。

2 全局结构：三层、一个协调点、七个底层模块



图中的箭头表示数据和调用方向。项目内部的合法依赖是 KnightUI.exe → KnightGame.dll → 底层 DLL。所有底层模块只与业务层交换共享数据；图中并排的底层模块之间没有调用关系。

层级	交付文件	负责的工作	不负责的工作
界面与交互层	<code>knightUI.exe</code>	显示菜单、地图、角色和状态；接收键盘、鼠标与窗口事件；播放画面和音效	不计算碰撞、伤害和敌人决策；不直接写存档
业务逻辑层	<code>knightGame.dll</code>	维护菜单与游玩会话；整理输入；决定每一帧的调用顺序；合并模块结果；处理关卡、复活和存档时机	不持有图形界面对象；不执行底层算法或 SQL
底层模块层	七个独立 DLL	分别管理世界、地图、运动、战斗、敌人、关卡内容和存档	彼此不链接、不调用；不自行决定跨模块业务流程

`knightGame.dll` 中的**游戏协调器**是唯一的跨模块汇合点。例如，运动模块把建议位移交给协调器，协调器再把它作为参数传给地图模块；地图模块并不知道运动模块是否存在。业务层最终把本帧所有候选变化交给状态模块统一提交。

七个底层模块各管一件事

中文模块名	动态库	对整个项目提供的能力	最重要的边界
对象与状态	<code>knightState.dll</code>	创建世界，保存运行中的实体状态，输出只读世界快照，统一提交状态变化	唯一拥有运行中世界的权威状态
地图与碰撞	<code>knightMap.dll</code>	建立地图索引，裁决墙壁与平台阻挡，报告危险区和交互区命中	不直接改变玩家位置或生命
运动与物理	<code>knightMotion.dll</code>	根据意图、重力和规则提出位移；根据碰撞结果形成最终运动候选	不直接读取键盘或调用地图模块
战斗与陷阱	<code>knightCombat.dll</code>	推进攻击阶段，判断命中、陷阱伤害、无敌时间和死亡候选	不直接扣除权威生命值
敌人决策	<code>knightAI.dll</code>	根据世界快照决定巡逻、索敌、追击和攻击意图	不直接移动敌人或调用战斗模块
关卡与配置	<code>knightContent.dll</code>	读取并完整校验图块、出生点、对象原型、目标、出口和资源引用	不创建运行中实体，不存玩家进度
存档与数据库	<code>knightSave.dll</code>	建库、读档、列出槽位、事务写入可恢复进度	不读取世界私有对象，不决定何时保存

3 数据归属：关卡定义、运行世界、持久进度

关卡配置、运行状态与存档

游戏协调器负责读取、汇总与传递



数据	由谁拥有	典型内容	什么时候变化
静态关卡定义	knightContent.dll 读取并校验；业务层持有返回的值对象	图块、出生点、敌人和陷阱原型、目标、出口	加载或切换关卡时
运行中的世界	knightState.dll 独占	玩家和敌人的位置、生命、攻击实例、当前检查点和机关状态	每个逻辑帧经一次统一提交
可恢复的进度	knightSave.dll 持久化；业务层从已提交世界提取副本	当前关卡、检查点、关键物品、能力、永久机关和目标进度	检查点、通关、主动保存时
界面显示数据	knightGame.dll 根据已提交世界制作；界面只读取	可绘制实体、状态栏、提示和一次性事件	关卡进入与逻辑帧提交后

共享的数据类型统一在 `knightContracts.h` 中定义。它是一个公共头文件，不生成 DLL。模块之间传的是命令、只读快照、运动和碰撞结果、状态变更候选、事件、关卡定义与存档副本；不会传递其他模块的私有对象、数据库连接或 Qt 图形对象。

需要持续遵守两条规则：**画面不是玩法状态的来源**，因此绘图坐标不能反写到世界；**存档不是实时世界的镜像**，因此每帧位置、速度和攻击实例不写入数据库。

4 一次操作如何变成游戏结果



每个逻辑帧按固定的六十分之一秒推进。界面定时器只负责唤醒，业务层根据真实经过时间决定是否执行玩法帧。整体顺序为：

- 1. 界面把按键或按钮转换成动作命令；业务层判断当前是主菜单、游玩中、暂停、死亡还是过关。
- 2. 业务层读取状态模块给出的旧世界快照，整理玩家按住状态和新按下动作；敌人决策模块返回敌人意图。
- 3. 运动模块给出建议位移，地图模块裁决阻挡和触发区，运动模块根据裁决形成最终运动候选。
- 4. 战斗模块计算攻击和陷阱候选；业务层判断拾取、检查点、门与出口，并合并同帧结果。
- 5. 状态模块**一次性提交**本帧变更。提交成功后，业务层决定死亡、过关和存档，制作画面快照与事件供界面显示。

所有底层计算结果在提交之前都只是**候选**。伤害导致死亡时，同一帧的检查点和出口候选取消；提交失败时不发布命中、拾取等成功事件。持续移动与一次攻击分别记录，按住移动键不会吞掉随后按下的攻击键。具体操作的逐步流程和函数名见[业务逻辑与数据流](#)。

与正常逻辑帧不同的三个流程

流程	业务层的处理原则
开始、读档、切关	先取得完整且已校验的关卡，再建立候选世界与候选地图；全部成功后一次切换，失败则释放候选并保留原会话
暂停与失焦	暂停时不推进世界时间；窗口失焦清空按住输入，恢复时重置计时基准
保存与返回菜单	只从已提交的世界制作存档副本；数据库确认成功后才释放当前关卡。保存失败保留会话，允许重试

5 工程如何构建与部署

项目以根目录 `CMakeLists.txt` 为构建入口。Qt Creator 和 Visual Studio 2022 都可以打开同一个 CMake 项目；编辑器可以不同，**共享的源文件、公开头文件和 CMake 目标必须相同**。各目标统一使用 Qt 6.12 对应的 MSVC2022 64 位套件，不混用 MinGW、不同位数或不同构建配置。

文件种类	项目中的作用	由什么生成、供谁使用
<code>.h</code>	共享结构体和公开函数声明	开发者维护；其他模块在编译时包含
<code>.cpp</code>	模块内部数据与函数实现	开发者维护；由编译器编译
<code>.lib</code>	Windows 上链接 DLL 使用的导入库	构建 DLL 时由链接器生成；上层目标在链接时使用
<code>.dll</code>	运行时执行的独立模块	构建各动态库目标生成；与程序一起部署
<code>.exe</code>	用户启动的游戏程序	构建界面目标生成；运行时加载业务和底层 DLL

项目内部的链接规则是：`knightUI.exe` 链接 `knightGame.lib`；`knightGame.dll` 链接七个底层模块各自的导入 `.lib`；任何底层模块都不链接另一底层模块的导入库。Qt 自身的框架库、运行时库和插件按所用功能部署；这不改变项目内部的三层依赖。

Qt 在本项目中负责窗口与事件循环、键鼠输入、二维绘制、计时唤醒及数据库支持；**玩家位置、攻击判定、碰撞结果和存档时机仍由项目自己的模块决定**。具体界面视觉稿可以晚于架构和接口确定。

建议把仓库按职责组织：`src/contracts` 放共享头文件，`src/ui` 放界面，`src/game` 放协调器，`src/state`、`src/map`、`src/motion`、`src/combat`、`src/ai`、`src/content`、`src/save` 分别放七个底层模块；`assets` 放关卡与音画资源，`tests` 放模块和流程检查，`docs` 放设计文档。`build` 只放生成文件，不作为人工修改的源码。

6 实施次序与两人协作方式

两人可以分别使用 Qt Creator 或 Visual Studio 编写代码，统一通过同一个仓库提交。分工以**模块所有权**为单位：每个模块由一名负责人维护其公开 `.h`、实现 `.cpp`、CMake 目标和模块检查；共享契约与跨模块接口变更由两人共同审阅。另一人的模块只通过公开接口使用，不复制其内部代码。

阶段	要完成的结果	可以进入下一阶段的条件
1. 确认契约	冻结共享结构体、公开函数签名、错误码和所有权规则	两人能按 模块接口协作约定 独立编写各自模块
2. 建立构建骨架	建立界面、业务和七个底层目标；所有公开函数有可链接的最小定义	CMake 能产出预期的 EXE、DLL 和导入 LIB；底层之间没有链接依赖
3. 贯通一关	菜单进入关卡，完成移动、跳跃、碰撞、攻击、受伤、检查点和存档	一条完整操作链能从界面命令追踪到世界提交与画面反馈
4. 扩展玩法	增加敌人行为、陷阱、物品门、多关切换、复活和读档	各模块仍只通过业务层交换数据；加载失败与保存失败有可恢复路径
5. 联合验收	按操作场景回归，构建部署可运行包	新机器可启动；存档可恢复；输入、碰撞、攻击和关卡流程符合文档

联调时先记录**命令** → **逻辑帧** → **模块输入/输出** → **唯一状态提交** → **画面与事件**。出现错误，按这条链定位到负责模块；不要在界面层或另一个底层 DLL 中绕过接口做临时修补。

7 架构验收清单

检查项	合格表现
模块隔离	七个底层 DLL 独立构建，链接关系中无底层到另一个底层的依赖
数据权威	运行中的世界只由状态模块提交；界面不反写坐标或生命
输入独立	按住移动键时，多次松开并重按攻击键，冷却完成后每次都能攻击
失败回滚	关卡加载失败不替换当前世界；保存失败保留当前会话和旧存档
游戏闭环	能完成开始、玩法、受伤/死亡、检查点、过关、存档、重新启动读档
构建部署	同一套 MSVC2022 64 位配置构建，EXE 与所需 DLL、Qt 运行时和插件齐全

本文件决定模块边界和协作顺序；[模块接口协作约定](#)决定参数、返回类型与所有权；[业务逻辑与数据流](#)决定具体操作中调用哪个模块、哪个函数及何时提交。实施者按这个顺序阅读即可从全局进入具体代码。

